

Testing AI-Generated Code

Why AI Code Needs More Testing, Not Less

Week 5 — Phase 2: Engineering

"Code that compiles is not code that works."

Instructor: Goker Ezberci

Vibe Coding 101: From Idea to Shipped Product

github.com/goker/vibe-coding-101-for-software-engineers



The False Confidence Problem

AI code compiles, runs, and looks professional. But is it correct?



Off-By-One

Page 1 starts at index 10 — users see empty first page



Hallucinated API

Imports a package that doesn't exist — npm install fails in prod



Happy-Path Only

Works for the demo, crashes on empty input or zero division



Silent Data Loss

json.dump fails on datetime — config file silently truncated

Pattern: Looks professional • Works for obvious case • Fails silently

Why AI Code Needs MORE Testing

The paradox: AI writes fast → you trust more → you test less → bugs ship

Without AI

- Write code slowly
- Think about edge cases while writing
- Small surface area — fewer functions
- Bugs in code you understand

With AI (No Tests)

- Generate code in seconds
- Skip thinking — it "just works"
- Huge surface area — many functions
- Bugs in code you only skimmed

Three Categories of AI Bugs

1. Confident-But-Wrong

Syntax correct, logic flawed

2. Plausible-But-Hallucinated

References things that don't exist

3. Works-For-Demo-Only

Happy path works, edge cases crash

The Testing Pyramid for AI Development

Classic layers + AI-specific foundation

E2E Tests

Does the deployed app work end-to-end?

↑ Fewer, slower, more expensive

Integration Tests

Do components work together?

Unit Tests

Does each function work correctly?

NEW

AI-Specific Tests

Hallucinations? Edge cases? Dependencies exist?

↓ More, faster, cheaper

AI-Specific Tests (The New Foundation)

Tests that only matter because AI generated the code



Hallucination Tests

Verify imported packages and APIs actually exist

```
assert importlib.import_module('json') # exists
assert importlib.import_module('intl_fmt') # FAILS
```



Edge Case Tests

Test the cases AI skips: empty, None, large input

```
assert get_tasks([]) == []
with pytest.raises(TypeError): get_tasks(None)
```



Boundary Tests

Test limits, zeros, negatives, extremes

```
assert get_page(items, 0) == items[:10]
with pytest.raises(ValueError): get_page(items, -1)
```



Why These Tests?

- ⚠ Human-written code rarely imports fake packages
- ⚠ AI optimizes for happy paths from training data
- ⚠ AI doesn't think about boundary conditions
- ❑ These tests catch the bugs UNIQUE to AI code
- ❑ Run them FIRST before traditional unit tests

TDD with AI: Red-Green-Refactor

You write the spec (test). AI writes the implementation.

1 RED

Write a failing test
(Human writes this)

2 GREEN

Ask AI to make it pass
(AI implements)

3 REFACTOR

Ask AI to clean up
(Tests stay passing)

```
# Step 1: YOU write the test (RED)
def test_add_task_returns_dict():
    result = add_task("Buy milk")
    assert result["text"] == "Buy milk"
    assert result["completed"] is False
    assert "id" in result
```

```
# Run: pytest → FAILS (Red!)
```

Step 2: Tell AI to make it pass (GREEN)

```
# Prompt to AI:
# "Implement add_task to make
# this test pass. Python stdlib
# only. Store in JSON file."
```

```
# AI implements → pytest → PASSES!
# Run: pytest → All green!
```



Tests are specifications. When you write tests first, you tell AI exactly what “correct” means.

TDD + AI vs. Traditional Coding

Why TDD is the perfect workflow for AI-assisted development

Aspect	Traditional Coding	TDD + AI
You think about...	Implementation details	Desired behavior
Code comes from...	Your typing	AI generation
If AI goes off-track...	You debug for hours	Tests catch it immediately
Design decisions...	AI's choice	YOUR choice (encoded in tests)
Confidence level...	"I think it works"	"Tests prove it works"
Time spent...	60% coding, 40% debugging	20% testing, 80% building
 Human = Specification Author • AI = Implementation Partner		

Evals: Testing the AI Itself

Tests verify your code. Evals verify your AI workflow.

Tests

Check:	Does this code work?
Run:	After code is written
On fail:	A specific function is broken
You fix:	The code

Evals

Check:	Does my AI workflow produce working code?
Run:	After AI generates code
On fail:	Your prompt, PRD, or instruction file
You fix:	Your AI configuration

Why Evals Matter

- Changed your CLAUDE.md? Did outputs get better or worse?
- Tweaked your PRD? Does AI still respect Non-Goals?
- Switched tools? Does the new one produce equivalent quality?

Three Levels of Evals

From basic output checks to regression tracking over time

1 Output Evals

Does the generated code meet the spec?

- ☐ Correct file structure?
- ☐ All tests pass?
- ☐ No extra dependencies?

Run: Per generation

2 Behavioral Evals

Does AI follow your rules?

- ☐ Respects Non-Goals?
- ☐ Follows Agent Rules?
- ☐ No hallucinated packages?

Run: Per PRD change

3 Regression Evals

Did your changes make things better or worse?

- ☐ Score before vs. after
- ☐ Track pass rate over time
- ☐ Compare across AI tools

Run: Over time

Building a Simple Eval Harness

Run AI on the same PRD N times, score the outputs, track pass rate

```
#!/bin/bash
# eval_run.sh - Simple eval harness
PASS=0; FAIL=0

for i in $(seq 1 5); do
  echo "=== Run $i ==="

  # 1. File structure correct?
  [ -f "run_$i/todo.py" ] && ((PASS++)) || ((FAIL++))

  # 2. Tests pass?
  cd "run_$i" && pytest -q && ((PASS++)) || ((FAIL++))

  # 3. No external dependencies?
  pip freeze > deps.txt
  [ $(wc -l < deps.txt) -eq 0 ] && ((PASS++))
done
echo "Pass rate: $PASS / $((PASS+FAIL))"
```



Metrics That Matter

Pass Rate

% of checks passed

Hallucination Rate

% with fake imports

Non-Goal Violations

Features AI shouldn't add

Coverage

% of code tested

Target

80%+ pass rate over 3-5 runs

Testing Tools (All Free)

Everything you need for professional testing at \$0

</> Python

pytest	Test runner + assertions
coverage.py	Line-by-line coverage reports
pytest-cov	Coverage integrated with pytest

```
pytest --cov=. --cov-report=term-missing
```

>_ JavaScript / TypeScript

vitest	Fast, Vite-native test runner
jest	Facebook's test framework
c8	V8 native coverage

```
npx vitest run --coverage
```



AI Can Write Tests — But You MUST Review Them

Rule: If you can't explain why a test would fail, it's not a real test.

Include: unit tests, edge cases (empty/None/large), integration tests, boundary tests. No over-mocking.

4 Testing Pitfalls to Avoid

AI creates these traps. Learn to spot them.

Tests That Pass By Definition

1

- `assert result is not None` # Too weak
- `assert result['text'] == 'test'` # Tests actual behavior

Over-Mocking Everything

2

- `@mock.patch('save')` # Mocks what you're testing
- `def test(tmp_path):` # Uses real filesystem

Testing Implementation, Not Behavior

3

- `uuid.UUID(result['id'])` # Breaks if ID strategy changes
- `assert t1['id'] != t2['id']` # Tests uniqueness

Eval Theater

4

- `'has_functions': 'def ' in code` # Trivially true
- `'tests_pass': run_tests(code)` # Actually verifies

The 5-Point Eval Checklist

Score each AI generation against these checks. Target: 80%+ pass rate across 3-5 runs.

1



Correct File Structure

Matches PRD phases and deliverables

2



All Tests Pass

Your test suite runs green on AI output

3



No Forbidden Dependencies

Matches PRD constraints — no surprise packages

4



No Non-Goal Violations

AI didn't add auth, database, or cloud sync

5



Agent Rules Respected

No features added beyond the PRD scope

Pass Rate = (checks passed) / (total checks across all runs) — Target: 80%+

Lab Exercise: Test + Eval Your Project

Part A: Test Suite

Add tests to your Week 3 project

- ☐ 4+ unit tests (one per core function)
- ☐ 3+ edge case tests (empty, None, boundary)
- ☐ 2+ integration tests (multi-step workflows)
- ☐ 1+ hallucination test (verify deps exist)
- ☐ 80% code coverage target

Part B: Mini Eval

Evaluate your AI workflow

- ☐ Run AI on your PRD 3 separate times
- ☐ Score each against 5-point checklist
- ☐ Calculate your pass rate
- ☐ Write: What did AI get wrong?
- ☐ Write: How to improve your PRD?



Push to your repo: tests/ directory, evals/ directory, updated README with testing section

Key Takeaways

- 1  AI code looks correct but hides bugs — test MORE, not less
- 2  TDD + AI: write the test first, let AI implement — you're the spec author
- 3  Add AI-specific tests: hallucination, edge case, and boundary tests
- 4  Evals test the AI itself — track pass rates when you change prompts or PRDs
- 5  Avoid eval theater — every check must be able to fail on real AI output